AyED 2025 Árboles y Heap - 1era fecha - 10/05/2025 Tema 4

Apellido	Nombre	Legajo	Corrigió
Ejercicio 1:	Ejercicio 2:	Ejercicio 3:	Total:

Ejercicio 1 (5 puntos).

Defina una clase **ParcialArboles** que contenga el siguiente método:

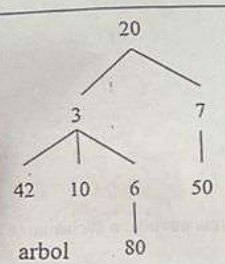
```
public static List<Integer> primerCaminoAlternanteCeroNoCero(GeneralTree<Integer>
arbol)
```

que recibe un árbol general de valores enteros y devuelve una lista con los valores de los nodos del **primer camino válido** desde la raíz hasta una hoja, tal que los valores de los nodos **alternen entre finalizados en cero y no finalizados en cero** en cada paso. Es decir, cada nodo del camino debe tener una condición opuesta a la de su padre: si un nodo tiene un número terminado en cero, su hijo inmediato debe tener un valor que no termine en cero y viceversa.

Se debe devolver el **primer camino completo** desde la raíz hasta una hoja que cumpla con esta condición, encontrado en un recorrido de izquierda a derecha.

Si no existe ningún camino desde la raíz hasta una hoja que cumpla con esta condición, el método debe devolver una lista vacía.

Ejemplo



1. 20→ 3 → 42
termina en cero → no termina en cero → no termina en cero
✕ No Alterna

2. 20 → 3 → 10
termina en cero → no termina en cero →termina en cero
✓ Alternancia válida

3. 20 → 3 → 6 → 80
termina en cero → no termina en cero → no termina en cero →termina en cero
✕ No Alterna

4. 20 → 7 → 50
termina en cero → no termina en cero →termina en cero
✓ Alternancia válida

- En un recorrido de izquierda a derecha, el camino 20 → 3 → 10 es el primero que se encuentra y el que tiene que retornar el método.

Observación importante:

- La alternancia puede comenzar desde cualquier valor en la raíz, ya sea que termine en cero o no. Lo importante es que a partir del segundo nodo (el primer hijo), los valores deben alternar la condición respecto al valor del nodo padre.
  - Por ejemplo, si la raíz del árbol fuera 2 (no termina en cero), un hijo con valor 30 (termina en cero) permitiría un inicio de camino alternante válido.

Tenga en cuenta que:

1. No puede agregar variables de instancia ni de clase a la clase **ParcialArboles.**
2. Debe respetar la clase y la firma del método indicado.
3. Puede definir todos los métodos y variables locales que considere necesarios.
4. Todo método que no esté definido en la sinopsis de clases debe ser implementado.
5. El método debe terminar ni bien encuentre un camino válido

Ejèrcicio 2 (1.5 puntos).

a. ¿Cuál es la expresión postfija de la siguiente expresión infija: A+(B-C)*D/(E*F)?:

(a) ABC-DEF-*/*+
(b) ABC-DE*F/*+
(c) AB-CDEF/**+
(d) ABC-D*EF*/+

b.- Construya una Min-Heap a partir de la siguiente secuencia, usando el algoritmo **BuildHeap**, G P C B A E H I, ¿cómo queda el recorrido preorden de la heap?

(a) A, B, C, G, P, E, H, I
(b) A, B, E, G, P, C, H, I
(c) A, C, B, G, P, E, H, I
(d) A, B, E, G, P, C, I, H

d.- Considere las siguientes sentencias:

(i) Un árbol binario puede contener al menos 2^L nodos en el nivel L
(ii) Un árbol binario completo de altura H, contiene 2^L nodos en cada nivel L, hasta la altura (H-1)
(iii) La altura de un árbol general de grado K lleno puede expresarse como $H=\log_k (N+1) -1$, siendo N el número total de nodos
(iv) Un árbol general lleno de grado K y altura H, tiene k^L nodos en el nivel L

¿Cuál de las opciones es correcta?

(a) (i) y (ii)
(b) (i), (ii) y (iii)
(c) (ii) y (iii)
(d) (ii), (iii) y (iv)
(e) (ii) y (iv)

Ejercicio 3 (3.5 puntos).

a.- Construya una **Max-Heap** con las siguientes claves insertándolas de una en una en la estructura inicialmente vacía, muestre cada uno de los pasos seguidos para construirla

12 33 1 21 74 81 28 88 99

b.- En la Max-Heap, construida anteriormente, ejecute una operación de DeleteMax.